

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – SAST SIMULATION

Course Code:	CSA5802	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Static Application Security Testing (SAST) using synthetic data		
Student Name:	Krithika M	Reg. No.:	192521385
Section / Batch:	B – Tech IT	Date:	10:08:2026

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
SAST SIMULATION	Creative Things and concept grasping	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I Krithika M 192521385 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **Krithika M** _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Rubrics:

Criteria	Weightage	Awarded Marks	Total Marks (100)
Understanding of Concepts	25%		
Experimental Design	30%		
Use of Tools	20%		
Analysis of Results	15%		
Documentation	10%		
Total per Question	100 Marks		

— 100

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

AI-Powered Secure Code Review and Vulnerability Detection

Course: DevSecOps Engineering and Application Security

Assessment Focus: SAST Simulation using Synthetic Data

1. Real-World Problem

An e-commerce company develops a Python-based web application for customer login, product search and order processing. If developers introduce insecure coding practices, the source code may contain vulnerabilities such as SQL Injection, hard-coded credentials, insecure input handling and weak password protection. A Static Application Security Testing (SAST) simulation can analyze the source code before deployment and identify these issues.

2. Aim

To simulate Static Application Security Testing (SAST) on a synthetic e-commerce application and identify security vulnerabilities directly from source code, followed by secure code fixes and re-scanning.

3. Synthetic Data

The following records are completely fictional and are used only for demonstrating the security testing process.

User ID	Username	Role	Login Attempts	Order Status
U1001	arun	Customer	2	Completed
U1002	meena	Customer	1	Pending
U1003	ravi	Customer	3	Completed
U1004	priya	Admin	2	Processing
U1005	karthik	Customer	4	Completed

4. Vulnerable Synthetic Application

Create a file named app.py. The application intentionally contains insecure coding patterns for SAST demonstration.

```
import sqlite3

ADMIN_PASSWORD = "admin123"

def login(username, password):
    if username == "admin" and password == ADMIN_PASSWORD:
        return "Login Successful"
    return "Login Failed"
```

```
def search_product(product):
    conn = sqlite3.connect("shop.db")
    query = "SELECT * FROM products WHERE name = '" + product + "'"
    result = conn.execute(query)
    return result.fetchall()

def display_message(name):
    return "<h2>Welcome " + name + "</h2>"
```

5. SAST Simulation

SAST analyzes application source code without requiring the application to be executed. The simulated scanner searches the source file for predefined insecure patterns and reports the detected vulnerabilities.

1. Read the source-code file.
2. Search for insecure coding patterns.
3. Assign a severity level.
4. Record the vulnerable line or code pattern.
5. Generate a vulnerability report.
6. Apply secure coding fixes and re-run the scan.

6. SAST Scanner

Create another file named `sast_scanner.py`.

```
import re

with open("app.py", "r") as file:
    code = file.read()

vulnerabilities = []

if 'ADMIN_PASSWORD = "admin123"' in code:
    vulnerabilities.append(("HIGH", "Hard-coded Password", "ADMIN_PASSWORD"))

if 'SELECT * FROM products WHERE name = ' in code:
    vulnerabilities.append(("HIGH", "Potential SQL Injection", "search_product()"))

if 'return "<h2>Welcome " + name' in code:
```

```
vulnerabilities.append(("MEDIUM", "Unescaped User Input", "display_message()"))

print("SAST SCAN RESULTS")
print("-" * 50)

for i, item in enumerate(vulnerabilities, 1):
    severity, issue, location = item
    print(f"{i}. [{severity}] {issue}")
    print ("  Location:", location)

print ("Total vulnerabilities detected:", len(vulnerabilities))
```

7. Execute the SAST Scanner

Run the following command in the terminal:

```
python sast_scanner.py
```

8. SAST Vulnerability Report

ID	Vulnerability	Severity	Location
V01	Hard-coded Password	High	ADMIN_PASSWORD
V02	Potential SQL Injection	High	search_product()
V03	Unescaped User Input	Medium	display_message()

9. Recommended Secure Fixes

- Hard-coded Password:** Use a secure password-management mechanism and avoid storing plaintext credentials in source code.
- SQL Injection:** Use parameterized SQL queries instead of directly concatenating user input into SQL statements.
- Unescaped User Input:** Escape or safely encode user-controlled data before inserting it into HTML output.

10. Re-run SAST

After applying the secure coding changes, run the SAST scanner again. The second scan should show that the previously identified insecure patterns have been reduced or removed.

Stage	Detected Issues
Before Fix	Hard-coded Password, SQL Injection, Unescaped Input

After Fix	Previously detected issues reduced after secure coding changes
-----------	--

11. SAST Workflow

Source Code → SAST Scanner → Pattern Analysis → Vulnerability Detection → Severity Classification → Secure Code Fix → Re-scan → Reduced Vulnerabilities

12. Difference Between SAST and DAST

SAST	DAST
Analyzes source code	Tests a running application
Application does not need to run	Application must be running
Finds insecure coding patterns	Finds runtime/web vulnerabilities
Tests internally	Tests externally

13. Results

Thus, Static Application Security Testing (SAST) was successfully simulated using synthetic data on an e-commerce application. The simulated scanner analyzed the source code and identified hard-coded credentials, potential SQL injection and unsafe handling of user input. Secure coding approaches were then applied and the source code was re-scanned to verify the improvement.

14. Conclusion

The simulation demonstrates how SAST can be integrated into a DevSecOps workflow to detect security weaknesses early in the software development lifecycle. Finding and fixing issues before deployment helps developers produce more secure applications.